

Déroulement Manuel de l'Algorithme de Benders

Planification Robuste sous Incertitude

Auteur : Mathis Francine-Habas
Structure : LAAS-CNRS

Date : 29 mai 2026
Version : 1.0

1. Première configuration

Afin de simplifier le déroulement de l'algorithme de Benders, nous allons prendre, dans un premier temps, Δ_i constant $\forall i \in \llbracket 1, T \rrbracket$. Nous suggérerons également que les coûts de lancement c^P ainsi que les bénéfices b sont nuls.

1.1. Formulation du problème

Soit le jeu de données suivant :

$$T = 3 \quad (1)$$

$$\Gamma = 2 \quad (2)$$

$$\Delta_i = 2, \forall i \in \llbracket 1, T \rrbracket \quad (3)$$

$$c^I = 1\text{€} \quad (4)$$

$$c^B = 2\text{€} \quad (5)$$

$$c^P = 0\text{€} \quad (6)$$

$$b = 0\text{€} \quad (7)$$

$$\widehat{D}_{nominal} = [10, 20, 30] \quad (8)$$

On obtient le graphe ci-dessous :

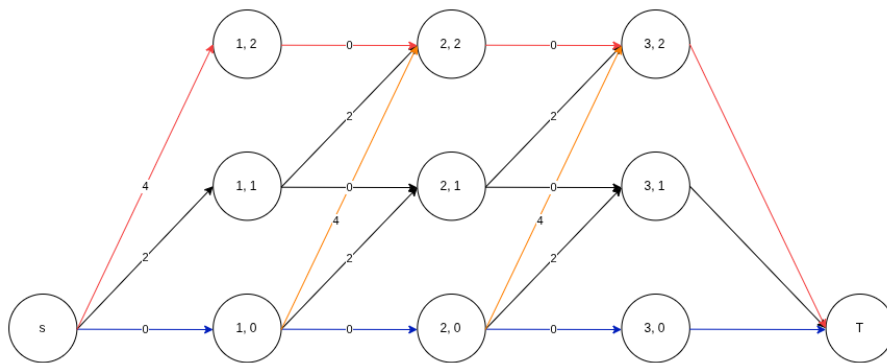


FIGURE 1 – Graphe initialisé avec les poids préliminaires

1.2. Itérations

Soit U l'ensemble contenant le scénario nominal et U' l'ensemble agrégé de U par les nouveaux scénarios remontés au Maître.

1.2.1 Step 1 : Itération naïve

Lower Bound (Problème Maître) Lors de la première étape, le problème Maître n'a aucune idée de l'incertitude, il ne connaît que le scénario nominal. Il va donc chercher à satisfaire $\widehat{D}_{nom} = [10, 20, 30]$.

- Décision du Maître : il va produire exactement la demande. Son plan de production cumulé est $X^{(1)} = [10, 20, 30]$;
- $LowerBound = 0€$ (car ni stock ni de rupture).

Upper Bound (Sous-Problème) L'adversaire observe le plan $X^{(1)}$ et va chercher à maximiser c^I et c^B tout en dépensant le maximum de budget possible $\Delta_i \leq \Gamma \forall i \in \llbracket 1, T \rrbracket$. Etant donné que la demande est cumulée, une attaque en $t = 1$ va faire beaucoup de dégâts. On a donc :

- L'adversaire lance l'attaque $\delta_1 = [+2, 0, 0]$;
- Nouveau scénario généré $D_{up} = [12, 22, 32]$;
- $UpperBound = 2 \times (2 + 2 + 2) = 12€$

Bilan itération 1 :

$$UB(12) > LB(0)$$

Nous ne sommes pas à l'équilibre. Le scénario D_{up} est remonté au Maître. On l'ajoute à U' .

1.2.2 Step 2 : Correction de l'attaque

Lower Bound Connaissant les scénarios contenus dans U' , le Maître doit surproduire pour se protéger de D_{up} , mais il sait également que s'il produit trop, le scénario nominal lui coûtera cher en frais de stockage. Il va donc chercher un point d'équilibre selon lequel le coût de surstock sera égal au coût de rupture face à l'attaque D_{up} .

L'objectif est donc de minimiser le coût Z . Posons $S = X_1 + X_2 + X_3$, la somme des décisions du plan d'attaque. Nous avons deux contraintes à satisfaire en même temps :

$$Z \geq c^I(X_1 - 10) + c^I(X_2 - 20) + c^I(X_3 - 30) \quad (9)$$

$$Z \geq (X_1 - 10) + (X_2 - 20) + (X_3 - 30) \quad (10)$$

$$Z \geq S - 60 \quad (11)$$

$$(12)$$

$$Z \geq c^B(12 - X_1) + c^B(22 - X_2) + c^B(32 - X_3) \quad (13)$$

$$Z \geq 2(12 - X_1) + 2(22 - X_2) + 2(32 - X_3) \quad (14)$$

$$Z \geq 132 - 2S \quad (15)$$

$$(16)$$

Minimiser Z revient à trouver l'intersection entre ces deux équations, aussi :

$$S = \frac{192}{3} = 64€$$

Or sachant que la demande nominale est de 60€, il nous reste 4€ à partager entre $\{X_1, X_2, X_3\}$. Si l'on décide de répartir ce stock de façon "lisse", en ajoutant une production supplémentaire constante α à chaque période, la marge s'accumule :

- $t = 1 : \alpha$;
- $t = 2 : 2\alpha$;
- $t = 3 : 3\alpha$.

En résolvant ce système d'équations, on obtient : $\alpha = \frac{2}{3} \approx 0.66$. Nous avons donc notre plan de production ajusté :

- $X_1 = 10 + \alpha = 10.66\text{€}$;
- $X_2 = 20 + 2\alpha = 21.33\text{€}$;
- $X_3 = 30 + 3\alpha = 32.00\text{€}$;
- $LowerBound = S - 60 = 4\text{€}$.

Upper Bound Puisque le Maître a sur-produit, l'adversaire va diminuer la demande, on a :

- L'adversaire lance l'attaque $\delta_2 = [-2, 0, 0]$;
- Nouveau scénario généré $D_{down} = [8, 18, 28]$;
- $UpperBound = 1 \times (2.66 + 3.33 + 4.00) = 10\text{€}$.

Bilan itération 2 :

$$UB(10) > LB(4)$$

Nous ne sommes pas à l'équilibre. Le scénario D_{down} est remonté au Maître.

1.2.3 Step 3 : Trouver l'équilibre

Lower Bound Le Maître connaît maintenant les deux extrêmes ($\in U'$). Il faut équilibrer les deux risques en fonction de leur coût respectif. Soit :

- Après résolution du système, on trouve le nouveau plan d'attaque $X^{(3)} = [10.33, 20.66, 31.00]$;
- $LowerBound = S - 54 = 8\text{€}$.

Upper Bound L'adversaire se heurte à un mur, le Maître est maintenant centré entre les pires bornes de l'incertitude. La pire attaque de l'adversaire plafonne à 8€ .

Bilan itération 3 :

$$UB(8) < LB(8) + \epsilon$$

Nous sommes à l'équilibre. L'algorithme a convergé. Graphiquement, il faudra vérifier que l'intérieur du graphe ($noeud(1,1)$, $noeud(2,1)$, $noeud(3,1)$) a été supprimé.

2. Seconde configuration

Dans cette seconde expérience, nous prendrons Δ_i inconstant $\forall i \in \llbracket 1, T \rrbracket$. Nous suggérerons de la même façon que les coûts de lancement c^P ainsi que les bénéfices b sont nuls.

2.1. Formulation du problème

Soit le jeu de données suivant :

$$T = 3 \tag{17}$$

$$\Gamma = 2 \tag{18}$$

$$\Delta_1 = 1 \tag{19}$$

$$\Delta_i = 2, \forall i \in \llbracket 2, T \rrbracket \tag{20}$$

$$c^I = 1\text{€} \tag{21}$$

$$c^B = 2\text{€} \tag{22}$$

$$c^P = 0\text{€} \tag{23}$$

$$b = 0\text{€} \tag{24}$$

$$\widehat{D}_{nominal} = [10, 20, 30] \tag{25}$$

On obtient le graphe suivant :

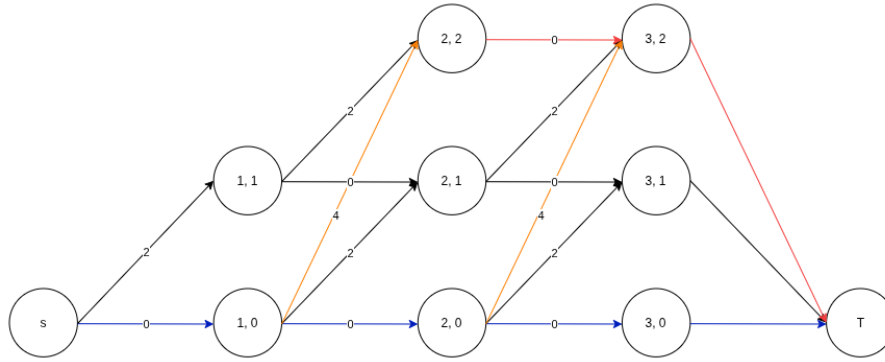


FIGURE 2 – Graphe initialisé avec les poids préliminaires de la seconde configuration

2.2. Itérations

2.2.1 Step 1 : Itération naïve

Lower Bound L'idée est la même concernant cette première étape.

- Plan de production cumulé : $X^{(1)} = [10, 20, 30]$;
- $LowerBound = 0€$.

Upper Bound De la même façon, une attaque en $t = 1$ va faire beaucoup de dégâts. On a donc :

- L'adversaire lance l'attaque $\delta_1 = [+1, +1, 0]$;
- Nouveau scénario généré $D_{up} = [11, 22, 32]$;
- $UpperBound = 2 \times (1 + 2 + 2) = 10€$

Bilan itération 1 :

$$UB(10) > LB(0)$$

Nous ne sommes pas à l'équilibre. Le scénario D_{up} est remonté au Maître.

2.2.2 Step 2 : Correction de l'attaque

Lower Bound L'objectif reste de minimiser le coût Z . Soit :

$$Z \geq c^I(X_1 - 10) + c^I(X_2 - 20) + c^I(X_3 - 30) \quad (26)$$

$$Z \geq (X_1 - 10) + (X_2 - 20) + (X_3 - 30) \quad (27)$$

$$Z \geq S - 60 \quad (28)$$

$$(29)$$

$$Z \geq c^B(11 - X_1) + c^B(22 - X_2) + c^B(32 - X_3) \quad (30)$$

$$Z \geq 2(11 - X_1) + 2(22 - X_2) + 2(32 - X_3) \quad (31)$$

$$Z \geq 130 - 2S \quad (32)$$

$$(33)$$

De plus,

$$S = \frac{185}{2} \approx 61.66$$

Il nous reste 1.66 à partager. On obtient : $\alpha = \frac{5}{9} \approx 0.555$. Nous avons donc notre plan de production ajusté $X^{(2)}$:

- $X_1 = 10 + \alpha = 10.55$;
- $X_2 = 20 + 2\alpha = 21.11$;
- $X_3 = 30 + 3\alpha = 31.66$;

Vérification et calcul de la Lower Bound :

- Si D_{nom} arrive, son surstock cumulé lui coûtera $c^I(0.55 + 1.11 + 1.66) = 3.33\text{€}$;
- Si D_{up} arrive, sa rupture cumulée lui coûtera $c^B(0.45 + 0.89 + 0.34) = 3.33\text{€}$.

Upper Bound L'adversaire va jouer le plan $X^{(2)}$ et attaquer à la baisse.

- L'adversaire lance l'attaque $\delta_2 = [-1, -1, 0]$;
- Nouveau scénario généré $D_{down} = [9, 18, 28]$;
- $UpperBound = (1.55 + 3.11 + 3.66) = 8.33\text{€}$

Bilan itération 2 :

$$UB(8.33) > LB(3.33)$$

Nous ne sommes pas à l'équilibre. Le scénario D_{down} est remonté au Maître.

2.2.3 Step 3 : Trouver l'équilibre

$$U' = \{D_{up} = [10.55, 21.10, 31.66], D_{nom} = [10, 20, 30], D_{down} = [9, 18, 28]\}$$

Lower Bound De la même façon, l'objectif est de chercher $X^{(3)}$ qui minimise le coût Z entre les deux scénarios les plus distants.

Après la résolution du système, nous trouvons le plan d'attaque cumulé $X^{(3)} = [10.28, 20.56, 30.83]$ tel que $Z = 6.66\text{€}$.

Upper Bound L'adversaire cherche la faille qui permettra d'infliger plus de 6.66€ mais le Maître s'est déjà paré contre toutes les attaques :

- S'il relance l'attaque D_{up} ou D_{down} le coût est de 6.66€ ;
- S'il tente une attaque hybride : par exemple, augmenter très fortement au milieu, la demande deviendrait $[10, 22, 32]$, le coût ne serait que de $5.22\text{€} \leq 6.66\text{€}$.

Bilan itération 3 :

$$UB(6.66) < LB(6.66) + \epsilon$$

Nous sommes à l'équilibre, il n'est plus possible de trouver des plans d'attaque qui dégraderont la valeur objective. Nous avons trouvé l'optimum.

2.3. Résultats de l'expérimentation par CPLEX

```

hand_benders_instances/parsed_instances/hand_instance_conf1.dzn_parsed 2 1
pi_value:124
pi_value:125.33333
pi_value:11.33335.33333
pi_value:13.33338
pi_value:8
STANDARD-done (Obj :8)
pi_value:12
8
pi_value:8
KC-----done (Obj :8)
pi_value:12
8
pi_value:8
KC_HOG---done (Obj :8)
Qualité valide

```

FIGURE 3 – Résultats pour les 3 méthodes étudiées sur la configuration 1.

```

hand_benders_instances/conf1/parsed_instances/hand_instance_conf2.dzn_parsed
pi_value:103.33333
pi_value:9.333335.33333
pi_value:9.333335.33333
pi_value:9.333336.66667
pi_value:6.66667
STANDARD-done (Obj :6.66667)
pi_value:10
6.66667
pi_value:6.66667
KC-----done (Obj :6.66667)
pi_value:10
6.66667
pi_value:6.66667
KC_HOG---done (Obj :6.66667)
Qualité valide

```

FIGURE 4 – Résultats pour les 3 méthodes étudiées sur la configuration 2.

Nous retrouvons les valeurs objectives souhaitées et constatons dès lors que les deux méthodes *KC* effectuent moins d'itérations que la méthode classique.

2.4. Etude des divergences sur les différentes méthodes

Pour bien comprendre les différences entre les trois algorithmes, nous reprendrons le cas d'étude 1. Le coût maximal étant de 4, nous avons plusieurs chemins possibles pour y parvenir :

$$[+2, 0, 0], [0, +2, 0], [0, 0, +2], [+1, +1, 0] \dots$$

2.4.1 Méthode Benders classique

L'idée est la suivante : "renvoyer et se protéger contre le premier scénario trouvé."

Exemple : Si le Maître trouve l'attaque $\delta = [+2, 0, 0]$ en premier, alors il va renvoyer le vecteur de demande associé $D' = [12, 22, 32]$. A la prochaine itération, le Maître va se blinder contre une grosse attaque à $t = 1$ mais restera vulnérable aux attaques sur $t = 2$ ou $t = 3$.

2.4.2 Méthode Knowledge Compilation original (KC)

Cette méthode va extraire (dans la matrice *arcbool*) tous les chemins qui ont un coût de 4. A la prochaine itération, il apprendra toutes les failles, mais son PLNE sera trop gros pour être résolu en temps fini.

2.4.3 Méthode KC avec heuristique d'orthogonalité (KC_HOG)

L'algorithme *DFS*¹ va trouver tous les chemins critiques, puis l'algorithme glouton (Max-min & Brays-Curtis) va filtrer.

- Il prendra le chemin $A = [2, 0, 0]$;
- Il cherchera les plus éloignés $B = [0, 2, 0]$ et $C = [0, 0, 2]$;
- Il rejettera $D = [1, 1, 0]$ car il est trop proche de A et B ;
- Enfin, il bouclera tant que le nombre de chemin sélectionnés est inférieur à $k > 0$ puis renverra *arcbool* allégé.

1. Algorithme de type Depth-First Search.